



BSc (Hons) in **Information Technology Specialized in AI**
IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.

3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`; ELSE `move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`

5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A mathematically perfect Table-Driven Agent is impractical for complex environments such as Chess because it would need a table containing an action for every possible percept sequence.

As the agent's lifetime increases, the length of the percept sequence also increases. This causes the number of possible percept sequences to grow extremely rapidly, resulting in combinatorial explosion. Therefore, the table would become extremely large and impossible to store and manage.

Instead of storing every possible percept sequence, we use an agent program with rules and internal logic to decide what action to take. The lab explains that the practical is based on replacing the impossible Table-Driven approach with agent programs.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
def sense_and_act(self, percept: dict) -> str:
```

```
    if percept['food_here']:
        return 'move_forward'
```

```
    elif percept['wall_ahead']:
        return 'turn_left'
```

```
    else:
        return 'move_forward'
```

These statements directly map to the rules of a Simple Reflex Agent:

```
IF food_here is True → THEN move_forward
ELSE IF wall_ahead is True → THEN turn_left
ELSE → move_forward
```

The agent only examines the current percept and immediately selects an action based on these condition-action rules. It does not maintain a history of previous percepts or actions.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent can get stuck because the environment is partially observable and the agent has no percept history. It only uses the current percept, such as whether there is a wall ahead or food at the current position. It does not remember previous actions or locations.

Therefore, when the agent reaches the same situation again, it makes the same decision again. For example, if there is a wall ahead, it always turns left. This can cause the agent to repeat the same sequence of actions and enter an infinite loop.

The main problem is that the agent cannot remember important information that is hidden from its current percept, such as where it has already been. Therefore, without percept history, it cannot recognize that it is repeating the same situation.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The Model-Based Agent uses an internal state to remember information about the environment. In my code, `last_action` stores the previous action, while `visited_cells` stores the cells that the agent has already visited.

The Transition Model is represented by updating the internal state according to the previous action. For example, turning left or right changes the agent's internal facing direction, while moving forward changes its estimated position.

The Sensor Model is represented by the percept received from the environment, such as `wall_ahead` and `food_here`. The agent uses this current percept together with its internal memory to make a decision.

Before selecting a new action, the Model-Based Agent updates its internal state using the previous action and percept. It can then check its memory and choose a different action if it detects that it has already visited a location or is repeating a situation. This allows it to avoid loops more effectively than the Simple Reflex Agent.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING